

IUT Paris cité Rives de Seine

Membres du binôme :

- **Ben Moumene Moussa (Groupe 111)**
- **Macy ARAB (Groupe 109)**

Objet du dossier : Développement d'un jeu de lettres "Octoverso" permettant à deux joueurs de former des mots à partir d'un chevalet de lettres et de marquer des points selon les mots joués, avec validation des mots via un dictionnaire et gestion des scores.

Table des Matières

I.	Page de garde	1
II.	Table des matières	1
III.	Présentation du projet	2
IV.	Graphe de dépendance des fichiers sources	2
V.	Bilan du projet	4
	a. Difficultés rencontrées	
	b. Réussites et fonctionnalités implémentées	
	c. Limites et améliorations possibles	
VI.	Annexes	5
	a. Code source complet	
	b. Tests effectués et résultats obtenus	
	c. Documentation technique (schémas, diagrammes, etc.)	

III. Présentation du projet 2

Ce projet consiste à développer un jeu de **Octoverso**, une version numérique du jeu classique inspiré de Scrabble. Le programme a pour objectif de simuler une partie complète entre deux joueurs, en respectant les règles du jeu et en utilisant une interface textuelle. L'application permet aux joueurs de saisir des mots à partir des lettres disponibles sur leur chevalet, de vérifier la validité des mots selon un dictionnaire intégré, et de jouer des mots sur un rail commun.

Chaque joueur commence avec un chevalet de 12 lettres générées aléatoirement. Le programme vérifie que chaque mot joué est valide à la fois par sa présence dans le dictionnaire et par la possibilité de le former avec les lettres du chevalet du joueur. Le jeu suit également les règles du placement des mots sur le rail, en veillant à ce qu'aucune collision ou erreur de placement n'intervienne. Le score des joueurs est mis à jour au fur et à mesure de la partie, et la fin de la partie est déterminée lorsque l'un des joueurs atteint un score maximal.

Les difficultés techniques de ce projet incluent la gestion des lettres aléatoires, la validation des mots avec un dictionnaire externe et l'organisation des mots sur le rail en respectant les contraintes de placement. Ce rapport présente l'architecture du projet, les tests unitaires réalisés pour valider chaque composant, ainsi qu'une trace d'exécution d'une partie complète du jeu.

IV. Graphe de dépendance des fichiers sources 2

L'application **Octoverso** a été architecturée autour de plusieurs composants clés : un **dictionnaire**, un **rail**, un **chevalet**, un **joueur** et un **jeu**.

Le composant **dictionnaire** gère toutes les opérations liées à la validation des mots (en vérifiant leur existence dans un fichier externe).

Le composant **rail** permet de manipuler et afficher les lettres sur le rail central du jeu.

Le **chevalet** est responsable de la gestion des lettres détenues par chaque joueur.

Le composant **joueur** représente chaque participant, stocke ses informations personnelles et son score, et gère son interaction avec le **chevalet**.

Enfin, le composant **jeu** coordonne la logique du jeu en contrôlant les tours des joueurs, la validation des mots, et la gestion de l'état du jeu.

Ce graphe permet de visualiser l'organisation interne du projet et de comprendre comment chaque composant interagit pour faire fonctionner **Octoverso**.

programmer soit au lycée soit avec mon binôme et aussi on avait au début un problème de compilation comme j'avais un mac et lui un Windows c'était compliqué mais ce problème n'a pas duré vu que le mac n'a pas finalement permis de faire le code

On a aussi très mal géré notre temps

b. Réussites et fonctionnalités implémentées

En terme de réussites on est content d'avoir réussi à mettre les lettres générées par ordre alphabétique

On s'est bien retrouvé dans le programme par rapport au fichier .c et .h même si c'était compliqué

Aussi une autre réussite c'est que on a réussi à mettre le dictionnaire.txt dans le programme

Une autre réussite et c'est pour cela qu'on aime la programmation c'est que après quelques jours de travaux et même de semaines on a réussi à faire fonctionner le 1^{er} tour on était très fier

Ce qui était bien c'est que lorsque on avait des erreurs soit d'inattention ou de liaison des fichiers on arrivait tout de même à se comprendre et à régler les problèmes

c. Limites et améliorations possibles

1. Raccourcissement des noms des fonctions

2. Faudrait qu'on améliore notre gestion de temps

3. Aussi une amélioration de la communication entre nous ce qui était dur lorsqu'on s'envoyait les codes soit moi soit mon binôme ne comprenait pas le code

4. Une meilleure relecture de notre programme nous aurait permis d'éviter certains problèmes d'inattentions ce qui nous aurait fait gagner du temps

5. Une meilleure répartition des tâches nous aurait permis de gagner du temps et de faire un meilleur travail

Main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "dictionnaire.h"
#include "chevalet.h"
#include "jeu.h"

// Verifié les lettres chevalet
bool verifier_lettre_chevalet(const char* mot, const char* lettres) {
    char temp_lettres[TAILLE_CHEVALET + 1];
    strcpy(temp_lettres, lettres);

    for (int i = 0; i < strlen(mot); i++) {
        char* pos = strchr(temp_lettres, mot[i]);
        if (pos == NULL) {
            return false;
        }
        *pos = ' ';
    }
    return true;
}

// Fonction pour générer un mot aléatoire à partir du dictionnaire
const char* generer_mot_aleatoire(const Dictionnaire* dict) {
    if (dict->taille == 0) {
        return NULL;
    }

    int index = rand() % dict->taille;
    return dict->mots[index];
}

int main() {
    srand(time(NULL));

    // Charger le dictionnaire
    Dictionnaire dict;
    if (!charger_dictionnaire(&dict, "dictionnaire.txt")) {
        printf("Erreur de chargement du dictionnaire\n");
        return EXIT_FAILURE;
    }
}
```

```

// Initialiser les joueurs et leurs chevaux
Chevalet chevalier_joueur1, chevalier_joueur2;
initialiser_chevalier_avec_lettres(&chevalier_joueur1);
initialiser_chevalier_avec_lettres(&chevalier_joueur2);

Joueur joueur1, joueur2;
initialiser_joueur(&joueur1, 1);
initialiser_joueur(&joueur2, 2);

joueur1.chevalier = chevalier_joueur1;
joueur2.chevalier = chevalier_joueur2;

// Affichage des chevaux des joueurs
printf(" 1 : ");
afficher_chevalier(&chevalier_joueur1);
printf(" 2 : ");
afficher_chevalier(&chevalier_joueur2);


char mot_joueur1[5], mot_joueur2[5];
while (1) { // Boucle principale
    // Joueur 1 choisit un mot
    do {
        printf("1> ");
        scanf("%4s", mot_joueur1);
    } while (strlen(mot_joueur1) != 4 ||
        !verifier_mot(&dict, mot_joueur1) ||
        !verifier_lettres_chevalier(mot_joueur1, chevalier_joueur1.lettres));

    // Joueur 2 choisit un mot
    do {
        printf("2> ");
        scanf("%4s", mot_joueur2);
    } while (strlen(mot_joueur2) != 4 ||
        !verifier_mot(&dict, mot_joueur2) ||
        !verifier_lettres_chevalier(mot_joueur2, chevalier_joueur2.lettres));

    printf("Joueur 1 a jouer le mot : %s\n", mot_joueur1);
    printf("Joueur 2 a jouer le mot : %s\n", mot_joueur2);

    // Condition pour sortir de la boucle
    char continuer;

```

```

        printf("Voulez-vous continuer ? (o/n): ");
        scanf(" %c", &continuer);
        if (continuer == 'n' || continuer == 'N') {
            break;
        }
    }

    return 0;
}

```

Joueur.c

```

#pragma warning (disable: 4996)
#include <stdio.h>
#include "dictionnaire.h"
#include "rail.h"
#include "chevalet.h"
#include "joueur.h"

// Initialiser un joueur
void initialiser_joueur(Joueur* joueur, int id) {
    joueur->id = id;
    initialiser_chevalet(&joueur->chevalet);
    joueur->score = 0;
}

// Afficher les informations du joueur
void afficher_joueur(const Joueur* joueur) {
    printf("Joueur %d:\n", joueur->id);
    printf("Score : %d\n", joueur->score);
    printf("Chevalet : ");
    afficher_chevalet(&joueur->chevalet);
}

// Fonction pour entrer un mot valide de taille donner
bool entrer_mot(Joueur* joueur, char* mot, int taille_mot) {
    printf("Joueur %d, entrez un mot de %d lettres : ", joueur->id, taille_mot);

    if (scanf("%s", mot) != 1) {
        printf("Erreur lors de la saisie du mot.\n");
        return false;
    }

    if (strlen(mot) != taille_mot) {

```

```

    printf("Erreur : Le mot doit avoir exactement %d lettres.\n", taille_mot);
    return false;
}

for (int i = 0; i < taille_mot; i++) {
    if (!contient_lettre(&joueur->chevalet, mot[i])) {
        printf("Erreur : La lettre '%c' n'est pas présente dans le chevalet.\n", mot[i]);
        return false;
    }
}

return true;
}

```

Jeu.c

```

#pragma warning (disable: 4996)
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#include "dictionnaire.h"
#include "jeu.h"
#include "rail.h"
#include "joueur.h"

// Fonction pour vérifier si un mot est valide (stub, lier au dictionnaire réel)
bool mot_valide(const char* mot) {

    return strlen(mot) > 0;
}

// Fonction pour jouer un mot sur le rail
bool jouer_mot(Jeu* jeu, const char* mot, bool a_droite) {
    Joueur* joueur = &jeu->joueurs[jeu->joueur_actuel];
    Joueur* adversaire = &jeu->joueurs[(jeu->joueur_actuel + 1) % 2];

    if (!mot_valide(mot)) {
        printf("Mot invalide !\n");
        return false;
    }

    for (const char* p = mot; *p != '\0'; p++) {

```



```

    if (!contient_lettre(&joueur->chevalet, *p)) {
        printf("Lettre '%c' manquante dans le chevalet !\n", *p);
        return false;
    }
}

if (!ajouter_au_rail(&jeu->rail, mot, a_droite)) {
    printf("Impossible d'ajouter le mot au rail (trop long) !\n");
    return false;
}

for (const char* p = mot; *p != '\0'; p++) {
    retirer_lettre(&joueur->chevalet, *p);
}

int lettres_expulsees = strlen(mot);
if (lettres_expulsees > jeu->rail.taille) {
    lettres_expulsees = jeu->rail.taille;
}

char expulsion[MAX_RAIL + 1] = { 0 };
if (a_droite) {
    for (int i = 0; i < lettres_expulsees; i++) {
        expulsion[i] = jeu->rail.lettres[i];
    }
    retirer_du_rail(&jeu->rail, lettres_expulsees, false);
}
else {
    for (int i = 0; i < lettres_expulsees; i++) {
        expulsion[i] = jeu->rail.lettres[jeu->rail.taille - lettres_expulsees + i];
    }
    retirer_du_rail(&jeu->rail, lettres_expulsees, true);
}

for (int i = 0; expulsion[i] != '\0'; i++) {
    ajouter_lettre(&adversaire->chevalet, expulsion[i]);
}

printf("Mot '%s' jou avec succ s ! Lettres expuls es : %s\n", mot, expulsion);
return true;
}

```

```

// Initialiser une partie
void initialiser_jeu(Jeu* jeu) {

    initialiser_rail(&jeu->rail);

    initialiser_joueur(&jeu->joueurs[0], 1);
    initialiser_joueur(&jeu->joueurs[1], 2);

    jeu->joueur_actuel = 0;
}

// Verifier si la partie est terminée
bool partie_terminee(const Jeu* jeu) {
    for (int i = 0; i < 2; i++) {
        if (jeu->joueurs[i].chevalet.taille == 0) {
            printf("\nLe joueur %d a gagn !\n", jeu->joueurs[i].id);
            return true;
        }
    }
    return false;
}

// Effectuer un tour de jeu
void jouer_tour(Jeu* jeu) {
    Joueur* joueur = &jeu->joueurs[jeu->joueur_actuel];

    // Afficher l'état actuel
    printf("\nC'est au tour du joueur %d :\n", joueur->id);
    afficher_joueur(joueur);
    afficher_rail(&jeu->rail);

    char mot[50];
    int taille_mot = 4;

    bool success = entrer_mot(joueur, mot, taille_mot);
    if (!success) {
        printf("Tour annulé, essayez encore.\n");
        return;
    }

    bool a_droite = true;

```

```

success = jouer_mot(jeu, mot, a_droite);
if (!success) {
    printf("Erreur lors du placement du mot sur le rail.\n");
    return;
}

// Passer au joueur suivant
jeu->joueur_actuel = (jeu->joueur_actuel + 1) % 2;

}

```

Rail.c

```

#pragma warning (disable: 4996)
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include "chevalet.h"
#include "rail.h"

// Initialiser le rail
void initialiser_rail(Rail* rail) {
    rail->taille = 0;
    for (int i = 0; i < MAX_RAIL; i++) {
        rail->lettres[i] = '\0';
    }
}

// Ajouter des lettres a une extremiter du rail
bool ajouter_au_rail(Rail* rail, const char* lettres, bool a_droite) {
    int nb_lettres = strlen(l lettres);
    if (rail->taille + nb_lettres > MAX_RAIL) {
        return false;
    }

    if (a_droite) {
        for (int i = 0; i < nb_lettres; i++) {
            rail->lettres[rail->taille + i] = lettres[i];
        }
    }
    else {
        for (int i = rail->taille - 1; i >= 0; i--) {

```

```

        rail->lettres[i + nb_lettres] = rail->lettres[i];
    }
    for (int i = 0; i < nb_lettres; i++) {
        rail->lettres[i] = lettres[i];
    }
}
rail->taille += nb_lettres;
return true;
}

// Retirer un nombre spécifier de lettres d'une extremiter du rail
bool retirer_du_rail(Rail* rail, int nb_lettres, bool a_droite) {
    if (nb_lettres > rail->taille) {
        return false;
    }

    if (a_droite) {
        for (int i = 0; i < nb_lettres; i++) {
            rail->lettres[rail->taille - 1 - i] = '\0';
        }
        rail->taille -= nb_lettres;
    }
    else {
        for (int i = 0; i < rail->taille - nb_lettres; i++) {
            rail->lettres[i] = rail->lettres[i + nb_lettres];
        }
        for (int i = rail->taille - nb_lettres; i < rail->taille; i++) {
            rail->lettres[i] = '\0';
        }
        rail->taille -= nb_lettres;
    }
    return true;
}

// Afficher le rail dans les deux sens (recto et verso)
void afficher_rail(const Rail* rail) {
    printf("Recto : ");
    for (int i = 0; i < rail->taille; i++) {
        printf("%c", rail->lettres[i]);
    }
    printf("\n");

    printf("Verso : ");
    for (int i = rail->taille - 1; i >= 0; i--) {
        printf("%c", rail->lettres[i]);
    }
    printf("\n");
}

```

```
}
```

Dictionnaire.c

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "jeu.h"
#include "dictionnaire.h"
#pragma warning (disable: 4996)

// Charger le dictionnaire a partir d'un fichier
bool charger_dictionnaire(Dictionnaire* dict, const char* fichier) {
    FILE* fp = fopen(fichier, "r");
    if (!fp) {
        perror("Erreur lors de l'ouverture du fichier");
        return false;
    }

    int taille = 0;
    char buffer[128];
    while (fgets(buffer, sizeof(buffer), fp)) {
        taille++;
    }

    rewind(fp);

    dict->mots = malloc(taille * sizeof(char*));
    if (!dict->mots) {
        perror("Erreur d'allocation mmoire");
        fclose(fp);
        return false;
    }

    dict->taille = taille;
    printf("Lecture du dictionnaire : %d mots\n", taille);
```

```

for (int i = 0; i < taille; i++) {
    if (fgets(buffer, sizeof(buffer), fp)) {

        buffer[strcspn(buffer, "\n")] = '\0';

        if (strlen(buffer) == 0) {
            printf("Avertissement : mot vide à la ligne %d\n", i + 1);
            continue;
        }

        dict->mots[i] = strdup(buffer);
        if (!dict->mots[i]) {
            perror("Erreur d'allocation de mémoire pour un mot");
            fclose(fp);
            return false;
        }
    }
}

fclose(fp);
return true;
}

// Verifier si un mot est dans le dictionnaire
bool verifier_mot(const Dictionnaire* dict, const char* mot) {
    for (int i = 0; i < dict->taille; i++) {
        if (strcmp(dict->mots[i], mot) == 0) {
            return true;
        }
    }
    return false;
}

// Libérer la mémoire du dictionnaire
void liberer_dictionnaire(Dictionnaire* dict) {
    for (int i = 0; i < dict->taille; i++) {
        free(dict->mots[i]);
    }
    free(dict->mots);
    dict->mots = NULL;
    dict->taille = 0;
}

```

```

void initialiser_historique(HistoriqueMots* historique) {
    historique->taille = 0;
    historique->capacite = 100;
    historique->mots_joues = malloc(historique->capacite * sizeof(char*));
}

bool ajouter_mot_joue(HistoriqueMots* historique, const char* mot) {
    // Vérifier si le mot est déjà joué
    if (historique->taille == historique->capacite) {
        historique->capacite *= 2;
        historique->mots_joues = malloc((historique->capacite) * (sizeof(char*)));
    }
    for (int i = 0; i < historique->taille; i++) {
        if (strcmp(historique->mots_joues[i], mot) == 0) {
            return false;
        }
    }
}

    historique->mots_joues[historique->taille++] = strdup(mot);
    return true;
}

// Libérer l'historique
void liberer_historique(HistoriqueMots* historique) {
    for (int i = 0; i < historique->taille; i++) {
        free(historique->mots_joues[i]);
    }
    free(historique->mots_joues);
    historique->taille = 0;
    historique->capacite = 0;
}

// Fonction pour vérifier si un mot est dans le dictionnaire
bool mot_valides(Dictionnaire* dict, const char* mot) {
    for (int i = 0; i < dict->taille; i++) {
        if (strcmp(dict->mots[i], mot) == 0) {
            return true;
        }
    }
    return false;
}

```

Chevalet.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "chevalet.h"

// Initialiser un chevalet
void initialiser_chevalet(Chevalet* chevalet) {
    chevalet->taille = 0;
    for (int i = 0; i < TAILLE_CHEVALET; i++) {
        chevalet->lettres[i] = '\0';
    }
}

// Ajouter une lettre au chevalet
bool ajouter_lettre(Chevalet* chevalet, char lettre) {
    if (chevalet->taille < TAILLE_CHEVALET) {
        chevalet->lettres[chevalet->taille++] = lettre;
        return true;
    }
    return false;
}

// Retirer une lettre du chevalet
bool retirer_lettre(Chevalet* chevalet, char lettre) {
    for (int i = 0; i < chevalet->taille; i++) {
        if (chevalet->lettres[i] == lettre) {
            for (int j = i; j < chevalet->taille - 1; j++) {
                chevalet->lettres[j] = chevalet->lettres[j + 1];
            }
            chevalet->taille--;
            return true;
        }
    }
    return false;
}

// Afficher le chevalet
void afficher_chevalet(const Chevalet* chevalet) {
    for (int i = 0; i < chevalet->taille; i++) {
        printf("%c ", chevalet->lettres[i]);
    }
    printf("\n");
}

```



```

}

// Vérifier si une lettre est dans le chevalet
bool contient_lettre(const Chevalet* chevalet, char lettre) {
    for (int i = 0; i < chevalet->taille; i++) {
        if (chevalet->lettres[i] == lettre) {
            return true;
        }
    }
    return false;
}

// Générer des lettres aléatoires pour le chevalet
void generer_lettres_aleatoires(char* lettres, int taille) {
    const char* alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    for (int i = 0; i < taille; i++) {
        lettres[i] = alphabet[rand() % 26];
    }
    lettres[taille] = '\0';
}

// Comparateur pour trier les lettres dans l'ordre croissant
int comparer_char_Croissant(const void* a, const void* b) {
    return (*(char*)a - *(char*)b);
}

// Trier les lettres du chevalet par ordre croissant
void trier_chevalet(Chevalet* chevalet) {
    qsort(chevalet->lettres, chevalet->taille, sizeof(char),
comparer_char_Croissant);
}

// Initialiser un chevalet avec des lettres aléatoires et les trier
void initialiser_chevalet_avec_lettres(Chevalet* chevalet) {
    generer_lettres_aleatoires(chevalet->lettres, TAILLE_CHEVALET);
    chevalet->taille = TAILLE_CHEVALET;
    trier_chevalet(chevalet);
}

// Vérifier si un mot peut être formé avec les lettres d'un chevalet
bool verifier_lettres_chevalet(const char* mot, const char* lettres) {
    char temp_lettres[TAILLE_CHEVALET + 1];
    strcpy(temp_lettres, lettres);

    for (int i = 0; i < strlen(mot); i++) {
        char* pos = strchr(temp_lettres, mot[i]);
        if (pos == NULL) {

```

```

        return false;
    }
    *pos = ' ';
}
return true;
}

```

Joueur.h

```

#pragma once
#pragma warning(disable: 4996)
#include <stdio.h>
#include "chevalet.h"

/**
 * @brief Structure représentant un joueur dans le jeu.
 */
typedef struct {
    int id;          /**< Identifiant unique du joueur (1 ou 2). */
    Chevalet chevalet; /**< Chevalet contenant les lettres du joueur. */
    int score;       /**< Score actuel du joueur. */
} Joueur;

/**
 * @brief Initialise un joueur avec un identifiant spécifique.
 * @param[out] joueur Le joueur à initialiser.
 * @param[in] id L'identifiant unique du joueur.
 */
void initialiser_joueur(Joueur* joueur, int id);

/**
 * @brief Affiche les informations du joueur, y compris son identifiant, son score, et son
        chevalet.
 * @param[in] joueur Le joueur dont les informations seront affichées.
 */
void afficher_joueur(const Joueur* joueur);

/**
 * @brief Permet à un joueur de saisir un mot en utilisant les lettres de son chevalet.
 * @param[in,out] joueur Le joueur qui saisit le mot.
 * @param[out] mot Le mot saisi par le joueur.
 * @param[in] taille_mot La taille maximale du mot que le joueur peut saisir.
 * @return true si le mot est valide et a été saisi avec succès, false sinon.
 * @pre Le mot doit être composé uniquement de lettres disponibles dans le chevalet du
        joueur.

```

```
*/  
bool entrer_mot(Joueur* joueur, char* mot, int taille_mot);
```

Jeu.h

```
#pragma once  
#pragma warning (disable: 4996)  
#include <stdio.h>  
#include <stdbool.h>  
#include <string.h>  
#include "rail.h"  
#include "joueur.h"  
#include "dictionnaire.h"  
  
typedef struct {  
    Rail rail;          // Rail principal  
    Joueur joueurs[2];  // Deux joueurs  
    int joueur_actuel;   // Index du joueur en cours (0 ou 1)  
    Dictionnaire dict;   // Dictionnaire des mots valides  
} Jeu;  
  
/**  
 * @brief Vérifie si un mot est valide en utilisant le dictionnaire.  
 * @param[in] mot Le mot à vérifier.  
 * @return true si le mot est valide, false sinon.  
 * @note Cette fonction est un stub et doit être liée à un dictionnaire réel.  
 */  
bool mot_valide(const char* mot);  
  
/**  
 * @brief Joue un mot sur le rail principal du jeu.  
 * @param[in,out] jeu Le jeu en cours.  
 * @param[in] mot Le mot à jouer.  
 * @param[in] a_droite Indique si le mot doit être joué à droite du rail.  
 * @return true si le mot a été joué avec succès, false sinon.  
 */  
bool jouer_mot(Jeu* jeu, const char* mot, bool a_droite);  
  
/**  
 * @brief Initialise une nouvelle partie de jeu.  
 * @param[out] jeu La partie de jeu à initialiser.  
 */  
void initialiser_jeu(Jeu* jeu);  
  
/**
```

```

* @brief Effectue un tour de jeu pour le joueur actuel.
* @param[in,out] jeu La partie en cours.
*/
void jouer_tour(Jeu* jeu);

/**
* @brief Vérifie si la partie est terminée.
* @param[in] jeu La partie en cours.
* @return true si la partie est terminée, false sinon.
*/
bool partie_terminee(const Jeu* jeu);

/**
* @brief Génère un ensemble de lettres aléatoires.
* @param[out] lettres Le tableau dans lequel les lettres générées seront stockées.
* @param[in] taille La taille du tableau de lettres à générer.
*/
void generer_lettres_aleatoires(char* lettres, int taille);

/**
* @brief Trie un tableau de lettres par ordre alphabétique.
* @param[in,out] lettres Le tableau de lettres à trier.
*/
void trier_lettres(char* lettres);

/**
* @brief Vérifie si les lettres d'un mot sont présentes dans un chevalet.
* @param[in] mot Le mot à vérifier.
* @param[in] lettres Les lettres disponibles dans le chevalet.
* @return true si le mot peut être formé avec les lettres du chevalet, false sinon.
*/
bool verifier_lettres_chevalet(const char* mot, const char* lettres);

```

Rail.h

```

#pragma once
#pragma warning (disable: 4996)
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#include "rail.h"
#include "joueur.h"

```

```
#include "dictionnaire.h"
```

```
typedef struct {
```

```
    Rail rail;          // Rail principal  
    Joueur joueurs[2];  // Deux joueurs  
    int joueur_actuel;   // Index du joueur en cours (0 ou 1)  
    Dictionnaire dict;   // Dictionnaire des mots valides  
} Jeu;
```

```
/**
```

```
 * @brief Vérifie si un mot est valide en utilisant le dictionnaire.  
 * @param[in] mot Le mot à vérifier.  
 * @return true si le mot est valide, false sinon.  
 * @note Cette fonction est un stub et doit être liée à un dictionnaire réel.  
 */
```

```
bool mot_valide(const char* mot);
```

```
/**
```

```
 * @brief Joue un mot sur le rail principal du jeu.  
 * @param[in,out] jeu Le jeu en cours.  
 * @param[in] mot Le mot à jouer.  
 * @param[in] a_droite Indique si le mot doit être joué à droite du rail.  
 * @return true si le mot a été joué avec succès, false sinon.  
 */
```

```
bool jouer_mot(Jeu* jeu, const char* mot, bool a_droite);
```

```
/**
```

```
 * @brief Initialise une nouvelle partie de jeu.  
 * @param[out] jeu La partie de jeu à initialiser.  
 */
```

```
void initialiser_jeu(Jeu* jeu);
```

```
/**
```

```
 * @brief Effectue un tour de jeu pour le joueur actuel.  
 * @param[in,out] jeu La partie en cours.  
 */
```

```
void jouer_tour(Jeu* jeu);
```

```
/**
```

```
 * @brief Vérifie si la partie est terminée.  
 * @param[in] jeu La partie en cours.  
 * @return true si la partie est terminée, false sinon.  
 */
```

```
bool partie_terminee(const Jeu* jeu);
```

```
/**
```

```
 * @brief Génère un ensemble de lettres aléatoires.
```

```

* @param[out] lettres Le tableau dans lequel les lettres générées seront stockées.
* @param[in] taille La taille du tableau de lettres à générer.
*/
void generer_lettres_aleatoires(char* lettres, int taille);

/**
* @brief Trie un tableau de lettres par ordre alphabétique.
* @param[in,out] lettres Le tableau de lettres à trier.
*/
void trier_lettres(char* lettres);

/**
* @brief Vérifie si les lettres d'un mot sont présentes dans un chevalet.
* @param[in] mot Le mot à vérifier.
* @param[in] lettres Les lettres disponibles dans le chevalet.
* @return true si le mot peut être formé avec les lettres du chevalet, false sinon.
*/
bool verifier_lettres_chevalet(const char* mot, const char* lettres);

```

Dictionnaire.h

```

#pragma once
#include <stdbool.h>
#include <stdio.h>

/**
* @brief Structure représentant un dictionnaire de mots.
*/
typedef struct {
    char** mots; /**< Tableau de pointeurs vers les mots du dictionnaire. */
    int taille; /**< Nombre de mots dans le dictionnaire. */
} Dictionnaire;

/**
* @brief Structure pour suivre les mots déjà joués au cours de la partie.
*/
typedef struct {
    char** mots_joues; /**< Tableau de pointeurs vers les mots déjà joués. */
    int taille; /**< Nombre de mots joués. */
    int capacite; /**< Capacité actuelle du tableau de mots joués. */
} HistoriqueMots;

```

```

/**
 * @brief Charge un dictionnaire à partir d'un fichier.
 * @param[out] dict Le dictionnaire à charger.
 * @param[in] fichier Le chemin du fichier contenant les mots du dictionnaire.
 * @return true si le dictionnaire a été chargé avec succès, false sinon.
 */
bool charger_dictionnaire(Dictionnaire* dict, const char* fichier);

/**
 * @brief Vérifie si un mot est présent dans le dictionnaire.
 * @param[in] dict Le dictionnaire à utiliser pour la vérification.
 * @param[in] mot Le mot à vérifier.
 * @return true si le mot est présent, false sinon.
 */
bool verifier_mot(const Dictionnaire* dict, const char* mot);

/**
 * @brief Libère la mémoire allouée pour un dictionnaire.
 * @param[in,out] dict Le dictionnaire dont la mémoire doit être libérée.
 */
void liberer_dictionnaire(Dictionnaire* dict);

/**
 * @brief Initialise un historique des mots joués.
 * @param[out] historique L'historique à initialiser.
 */
void initialiser_historique(HistoriqueMots* historique);

/**
 * @brief Ajoute un mot à l'historique des mots joués.
 * @param[in,out] historique L'historique auquel ajouter le mot.
 * @param[in] mot Le mot à ajouter.
 * @return true si le mot a été ajouté avec succès, false sinon.
 */
bool ajouter_mot_joue(HistoriqueMots* historique, const char*);

/**
 * @brief Vérifie si un mot est présent dans le dictionnaire.
 * @param[in] dict Le dictionnaire dans lequel rechercher le mot.
 * @param[in] mot Le mot à vérifier dans le dictionnaire.
 * @return true si le mot est trouvé dans le dictionnaire, false sinon.*
 */
bool mot_valides(Dictionnaire* dict, const char* mot);

```

Chevalet.h

```
//chevalet.h
#pragma once
#pragma warning(disable: 4996)
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define TAILLE_CHEVALET 7

/**
 * @brief Structure représentant un chevalet contenant des lettres.
 */
typedef struct {
    char lettres[TAILLE_CHEVALET + 1]; /**< Lettres présentes dans le chevalet (+1 pour le
    caractère nul). */
    int taille; /**< Nombre actuel de lettres dans le chevalet. */
} Chevalet;

/**
 * @brief Initialise un chevalet vide.
 * @param[out] chevalet Le chevalet à initialiser.
 */
void initialiser_chevalet(Chevalet* chevalet);

/**
 * @brief Ajoute une lettre au chevalet.
 * @param[in,out] chevalet Le chevalet auquel ajouter la lettre.
 * @param[in] lettre La lettre à ajouter.
 * @return true si la lettre a été ajoutée avec succès, false sinon.
 */
bool ajouter_lettre(Chevalet* chevalet, char lettre);

/**
 * @brief Retire une lettre du chevalet.
 * @param[in,out] chevalet Le chevalet dont la lettre sera retirée.
 * @param[in] lettre La lettre à retirer.
 * @return true si la lettre a été retirée avec succès, false sinon.
 */
bool retirer_lettre(Chevalet* chevalet, char lettre);

/**
```



```

* @brief Affiche les lettres du chevalet.
* @param[in] chevalet Le chevalet à afficher.
*/
void afficher_chevalet(const Chevalet* chevalet);

/**
* @brief Vérifie si une lettre est présente dans le chevalet.
* @param[in] chevalet Le chevalet à vérifier.
* @param[in] lettre La lettre à rechercher.
* @return true si la lettre est présente, false sinon.
*/
bool contient_lettre(const Chevalet* chevalet, char lettre);

/**
* @brief Génère un ensemble de lettres aléatoires.
* @param[out] lettres Le tableau dans lequel les lettres générées seront stockées.
* @param[in] taille La taille du tableau de lettres à générer.
*/
void generer_lettres_aleatoires(char* lettres, int taille);

/**
* @brief Trie les lettres présentes dans le chevalet par ordre alphabétique.
* @param[in,out] chevalet Le chevalet dont les lettres doivent être triées.
*/
void trier_chevalet(Chevalet* chevalet);

/**
* @brief Initialise un chevalet avec des lettres aléatoires, puis les trie.
* @param[out] chevalet Le chevalet à initialiser.
*/
void initialiser_chevalet_avec_lettres(Chevalet* chevalet);

```